

Table of Contents

University Examination Management System

Final Software Engineering Documentation

Document date: 28 September 2026

Project type: University software engineering project

Language of application interface: Persian

Document status: Consolidated project documentation

Evidence and interpretation

This document consolidates the approved requirements, the database design, the reported successful Prisma initial_schema migration, and the described backend and frontend modules. The application source repository and running deployment were not available for direct inspection during preparation. Consequently, diagrams describe the agreed domain and module boundaries, not independently verified source code. API routes below are resource families and illustrative contracts; exact paths and payloads must be checked against the repository before publication as an API reference. The database design artifact calls itself a design rather than a deployed migration, while a subsequent project transcript reports a successful migration. A seven day objection example in that artifact conflicts with the later approved six month objection period. Six months is the governing requirement here; its implementation must be checked.

1 Introduction and problem statement

The system supports university examinations from course setup through participation, grading, publication and grade objections. Students and professors need a single controlled workflow for multiple choice and descriptive exams. Manual or fragmented methods can obscure eligibility, deadlines, submissions, grading decisions and review history. The system records these events under role and course ownership rules.

2 Objectives and scope

The objectives are to let eligible students take scheduled exams, allow assigned professors to prepare and grade them, enforce deadlines on the server, publish grades deliberately, support review requests, and retain meaningful grade history. The administrative setup role provisions accounts and academic relationships. The scope covers a Persian interface, Solar Hijri presentation, course and exam management, two exam types, private descriptive files, grades, objections and student performance views. It does not claim self registration, online proctoring, notifications, accommodations or automated marking of descriptive answers.

3 Stakeholders and actors

Actor or stakeholder	Interest and responsibility
Student	Takes authorized exams; checks submissions and published grades; submits objections.
Professor	Manages assigned courses and exams; reviews submissions; publishes grades; decides objections.
Administrator	Provisions accounts, courses, enrollments and professor assignments; supports setup.
University academic office	Defines schedules, assessment policy and access responsibilities.
System operator	Runs database, application, private file storage and backups.

4 Functional requirements

1. University provisioned users authenticate with their assigned role; no public registration is offered.
2. Admin setup maintains course offerings, student enrollments and professor assignments.
3. Professors manage exams only through their assigned courses. Exams have a type, schedule, possible duration, questions and score allocation.
4. Students see only published exams for courses in which they are eligible, subject to time rules.
5. A multiple choice attempt supports numbered question navigation, selecting and saving answers, final submission and backend scoring. The question response excludes correct answer flags.
6. A descriptive attempt supports authorized question file download and answer upload as PDF, JPG, JPEG or PNG, followed by final submission.
7. Deadline processing preserves answered multiple choice questions, records unanswered ones as unanswered, and records a descriptive non submission with no accepted file as absent.
8. Professors inspect submissions, review descriptive answer files, assign scores and publish grades. Students see only published grades.
9. Students can inspect course and exam results, comparisons and trends; they can submit an objection during the approved six month window and see its status and response.
10. A professor may confirm or revise a grade with a reason and response. Material revisions retain old and new scores.

5 Nonfunctional requirements and assumptions

Area	Requirement or assumption
Security	Server side authorization on every protected resource; least privilege and private file access.
Integrity	Transactional finalization and idempotent handling of repeated submissions; scores computed server side.
Time	Store absolute instants; display and accept Solar Hijri dates in the Persian interface; compare deadlines using trusted server time and Asia/Tehran conversion at the boundary.
Usability	Persian right to left interface, clear progress and confirmation, accessible labels and errors.
Reliability	Durable PostgreSQL records, recovery backups and observable errors without leaking secrets.
Performance	Indexed eligibility and ownership lookups; pagination for lists and bounded upload sizes. No numeric service level target has been approved.
Assumptions	University supplies accounts and academic assignments; network and private file storage are available; a student has at most one attempt per exam in the designed schema.

6 Business rules

- A student must have an active enrollment for the exam course; a professor must hold the applicable assignment. An admin setup action does not remove the exam's professor assignment requirement.
- A draft exam can be edited; publication requires valid scheduling, questions and score configuration. Published exam changes must respect existing attempts.
- An attempt deadline is the earlier of the scheduled end and the individual's duration end when a duration applies. Finalization after the cutoff is rejected or performed by the deadline process according to attempt type.

- At MCQ cutoff, saved answers count and missing answers remain unanswered. A descriptive attempt without an accepted file at cutoff is absent. A descriptive final submission requires an accepted answer file.
- Correct option flags and answer keys stay out of student question payloads before submission. An attempt cannot be modified after finalization.
- Grade publication is an explicit transition. An objection is available only against a published grade, within the approved six month period. The precise interpretation of a calendar month and subsequent review rounds needs reconciliation with current code and policy.
- The database design proposes private answer file byte retention and later purge after six months while retaining metadata. Treat this as a design policy requiring verification, separate from the objection window.

7 Use cases

ID	Use case	Primary actor	Successful outcome
UC01	Sign in	Student, Professor, Admin	Authenticated session with role.
UC02	Configure course relationships	Admin	Course, enrollment or assignment saved.
UC03	Prepare and publish exam	Professor	Eligible students can see a valid published exam.
UC04	Take MCQ exam	Student	Answers finalized and score calculated.
UC05	Submit descriptive exam	Student	Accepted files linked to finalized attempt.
UC06	Review and publish grade	Professor	Published result visible to student.
UC07	Object to grade	Student	Objection recorded within window.
UC08	Resolve objection	Professor	Response and any revision recorded.

Detailed use cases

UC01 Sign in. Preconditions: active university provisioned account. The actor enters credentials; the server verifies the password and account state, issues access and refresh credentials, and returns a role scoped identity. Invalid credentials receive a generic failure; logout or token rotation invalidates the affected refresh credential.

UC03 Prepare and publish exam. Preconditions: professor authentication and course assignment. The professor sets exam type, schedule, questions or question file, points and, for MCQ, options and exactly one correct answer per question. Validation precedes publication. A draft lacking required data stays unpublished; an unassigned professor is denied.

UC04 Take MCQ exam. Preconditions: published exam, active enrollment and open time window. The server creates or retrieves the student's single attempt and immutable question order. The student navigates by question number, saves choices and confirms submission. The server checks selected options belong to their questions, locks the attempt and calculates the score. At cutoff, saved answers are scored and missing answers remain unanswered. Late writes, duplicate attempts and cross student access are rejected or safely resolved to the existing final state.

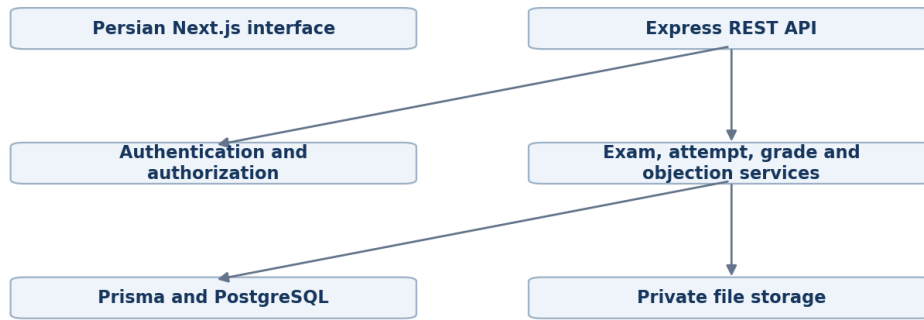
UC05 Submit descriptive exam. Preconditions: same eligibility and timing checks. The student downloads any question file through an authorized endpoint, uploads allowed answer files to private storage, and confirms final submission. The server validates file type and limits, links the file to the student's attempt and finalizes before cutoff. With no accepted file at deadline, the attempt becomes absent; unauthorized or late uploads fail.

UC06 Review and publish grade. Preconditions: professor owns the exam. The professor reads the authorized submission and private answer file, assigns a score within exam bounds with comments where supported, and publishes the grade. Draft grades remain hidden from students.

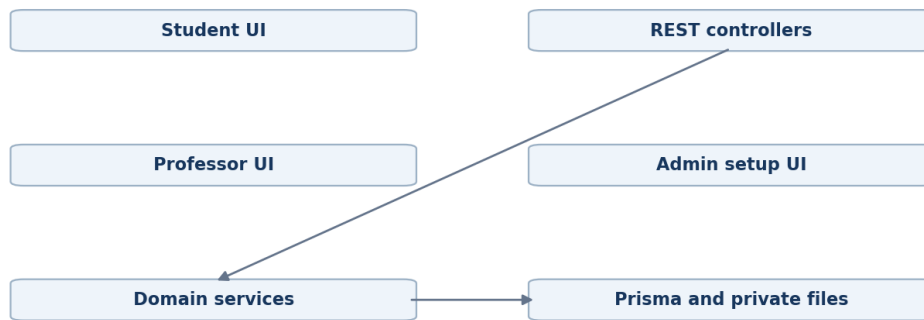
UC07 and UC08 Objection review. A student with a published grade submits a reason within six months of publication. The assigned professor opens it, reviews the original result, writes a response and confirms or changes the score. A change creates an append only revision with old score, new score, reason and actor; the student sees the result and response. Unauthorized actors and requests outside the period fail.

8 System architecture and technology stack

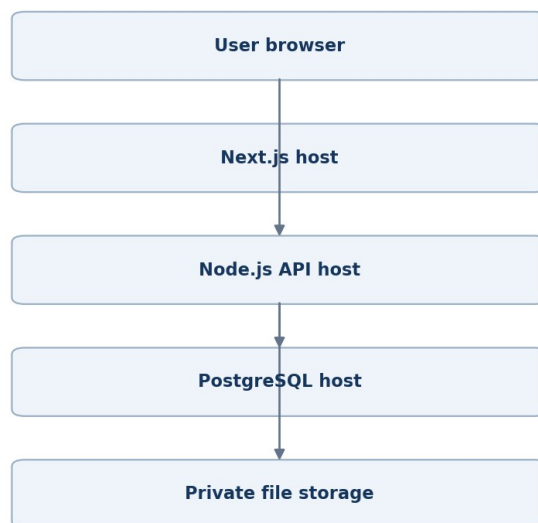
The architecture separates a Persian Next.js client from a Node.js, Express and TypeScript REST API. Middleware performs authentication, role checks, input validation and error handling; services enforce ownership, deadlines and transitions; Prisma accesses PostgreSQL. File bytes are held in private storage and served only through authorized handlers. JWT access and refresh credentials support sessions; password hashes are verified on the server. The schema design uses PostgreSQL timestamps with timezone and relational constraints.



Component diagram. UI modules cover student dashboard and exam taking, professor exam management and grading, and administrative setup. The API modules share identity and ownership checks; their database writes and file access are mediated through services.



Deployment diagram. This is a deployment requirement topology, not a claim of a deployed environment.



9 Database design and ER diagram

The reported initial Prisma migration succeeded after a one to one relation validation issue was corrected. The available design names User, Student, Professor, Course, Enrollment, ProfessorAssignment, Exam, ExamQuestion, MultipleChoiceOption, ExamAttempt, AttemptQuestion, StudentAnswer, StoredFile, Grade, GradeRevision, GradeObjection, ObjectionDecision and AuditEvent. The current migrated schema should be compared with that design before treating every field or constraint below as verified.

User identifies a university provisioned account. Course is a term specific offering, with a unique (code, termCode). Enrollment and ProfessorAssignment retain active flags and historical links. Exam references the assigned professor and course. An attempt is unique by (examId, studentId) and is tied to an enrollment for the same course. AttemptQuestion captures per attempt ordering; StudentAnswer points to an option for its placed question. Grade is unique per attempt. ObjectionDecision records a response; GradeRevision records a changed score. Foreign keys use restrictive deletion for historical academic records.

ENTITY	RELATIONSHIP	RELATED ENTITY
USER	has	STUDENT
USER	has	PROFESSOR
STUDENT	receives	ENROLLMENT
COURSE	includes	ENROLLMENT
PROFESSOR	holds	PROFESSOR_ASSIGNMENT
COURSE	grants	PROFESSOR_ASSIGNMENT
COURSE	contains	EXAM
PROFESSOR_ASSIGNMENT	owns	EXAM
EXAM	defines	EXAM_QUESTION
EXAM_QUESTION	offers	MULTIPLE_CHOICE_OPTION
ENROLLMENT	authorizes	EXAM_ATTEMPT
EXAM	has	EXAM_ATTEMPT
EXAM_ATTEMPT	orders	ATTEMPT_QUESTION
ATTEMPT_QUESTION	records	STUDENT_ANSWER
EXAM_ATTEMPT	holds	STORED_FILE
EXAM_ATTEMPT	earns	GRADE
GRADE	preserves	GRADE_REVISION
GRADE	receives	GRADE_OBJECTION
GRADE_OBJECTION	resolves	OBJECTION_DECISION

Class diagram

The class view shows domain relationships rather than framework classes or an assertion about source filenames.

CLASS	RELATIONSHIP	RELATED CLASS
User	enrollment or assignment	Course
Course	contains	Exam
Exam	receives	ExamAttempt
ExamAttempt	produces	Grade
Grade	challenged by	GradeObjection

10 API architecture and access policy

The REST API is grouped by auth, courses, exams, questions, attempts or submissions, files, grades and objections. Controllers validate input and invoke services; services check enrollment or professor assignment on the target record, not merely the caller's role. Responses to students omit isCorrect and unpublished grades. Upload and download endpoints check file ownership via the linked exam or attempt. Errors distinguish validation, unauthenticated, forbidden, missing resource, conflict and closed window conditions without exposing secrets. Exact route names, body schemas and status codes are implementation dependent and require repository verification.

Resource family	Student access	Professor access	Admin access
Courses and exams	Own enrollment and published exams	Assigned courses and owned exams	Setup as authorized
Attempts and files	Own attempt and eligible exam question file	Assigned exam submissions and files	No general grade editing implied
Grades	Own published results	Grades on assigned exams	No general grade editing implied
Objections	Own published grade objections	Objections on assigned exams	No general review authority implied

11 Authentication authorization and security

Login verifies a hashed password, issues short lived access and rotating refresh tokens, and protects logout and refresh flows against token reuse. API guards check authentication and role, then query ownership through enrollment, assignment, attempt and grade relationships. The client may hide unauthorized controls, but the API remains authoritative.

Use parameterized Prisma operations; avoid raw SQL assembled from input. Validate and sanitize displayed content against XSS, configure CORS to known origins and address CSRF

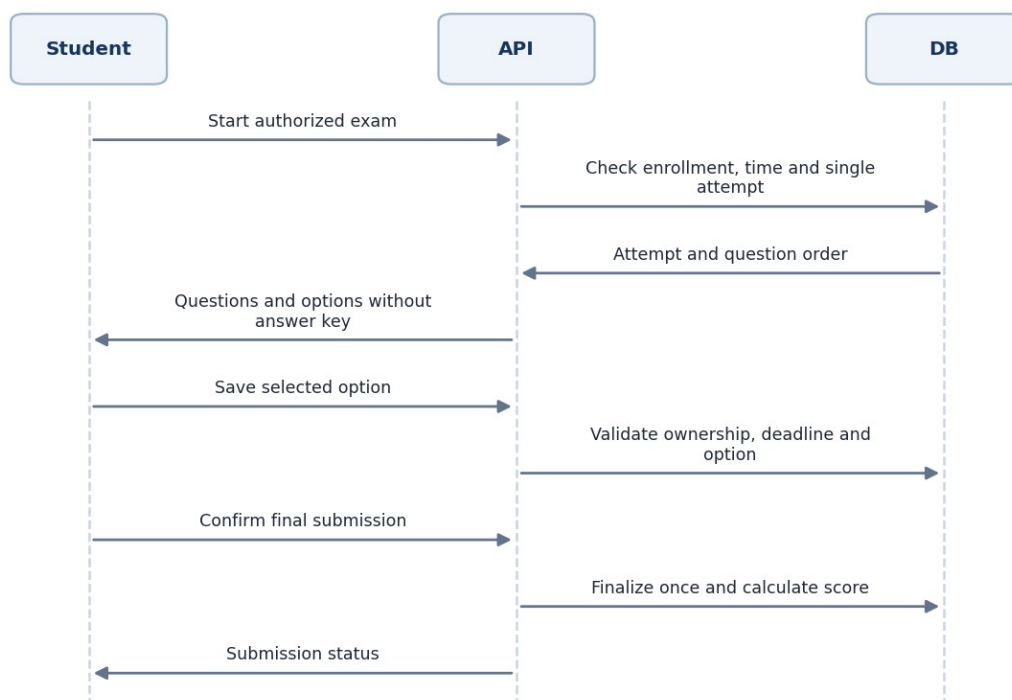
if browser credentials are cookie based. Rate limit login and sensitive routes. Keep secrets in environment configuration, redact tokens and personal data from logs, and avoid detailed authentication errors. File handlers enforce extension and MIME allowlists, content inspection where available, size limits, randomized storage keys and private storage; downloads resolve database identifiers rather than user controlled paths. Database transactions and unique constraints protect final submission and grade revision from races. Security tests must cover student to student and professor to professor IDOR, score tampering, early answer disclosure, malicious upload and unauthorized grade or objection changes.

12 Workflows

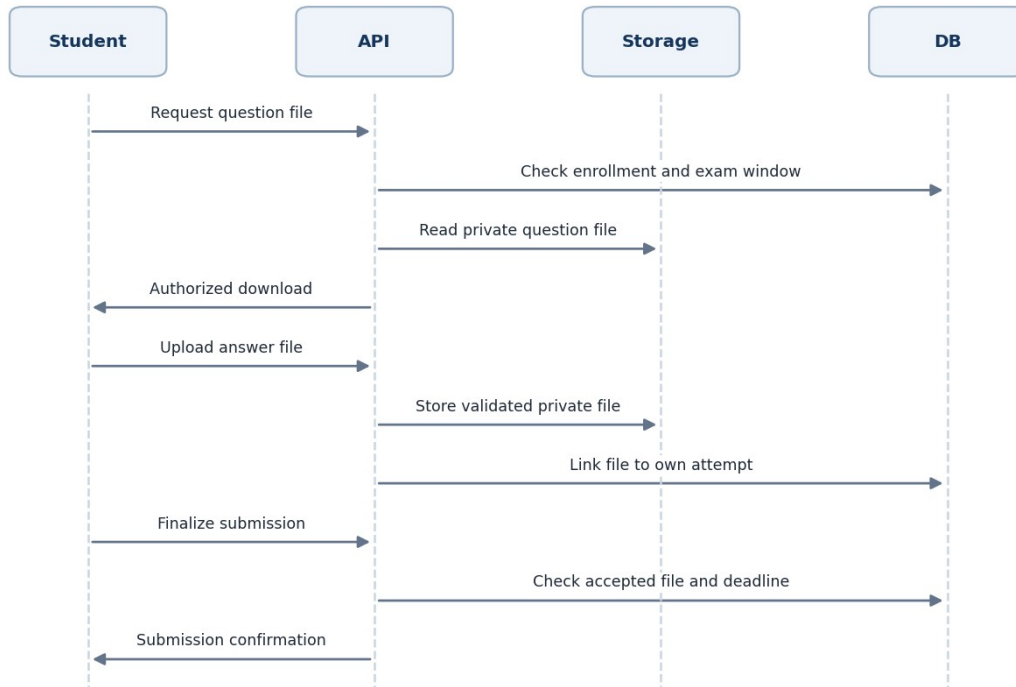
Student and professor workflows

A student signs in, sees course scoped upcoming, active and completed exams, starts an eligible attempt, submits, checks its status, reads a published grade and may object. A professor signs in, sees assigned courses, prepares and publishes an exam, reviews submissions, grades descriptive work, publishes grades and responds to objections. The admin establishes the accounts and academic links these workflows require.

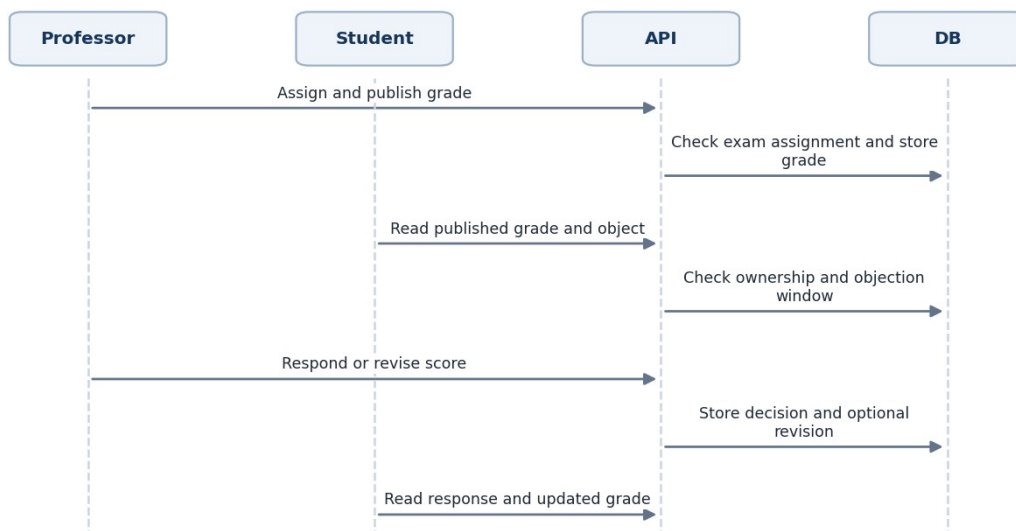
Multiple choice sequence diagram



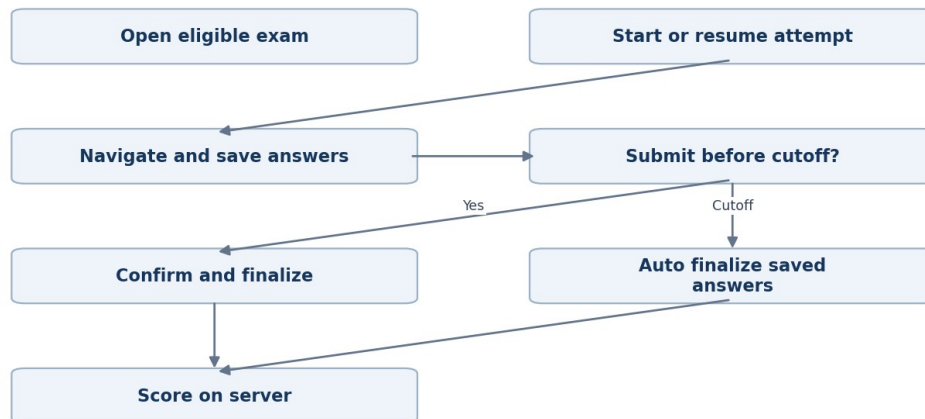
Descriptive sequence diagram



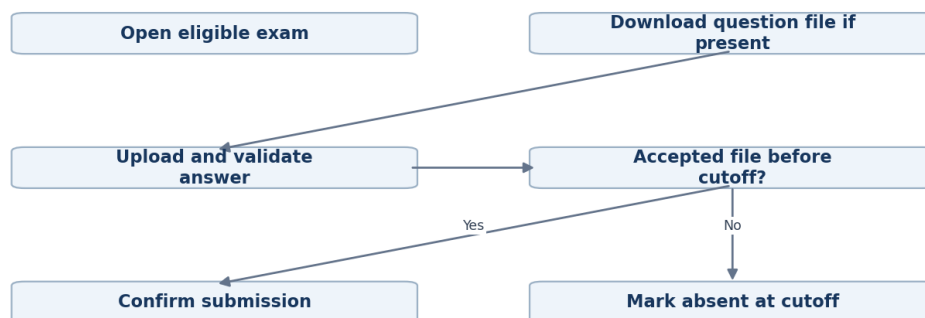
Grading and objection sequence diagram



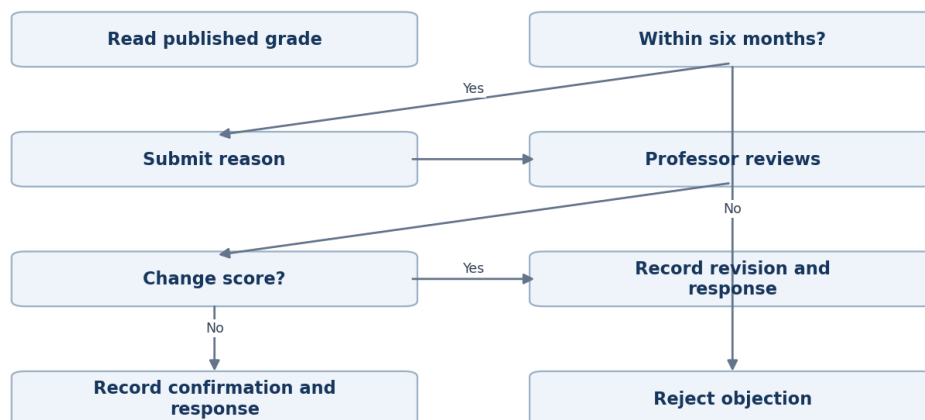
MCQ activity diagram



Descriptive activity diagram

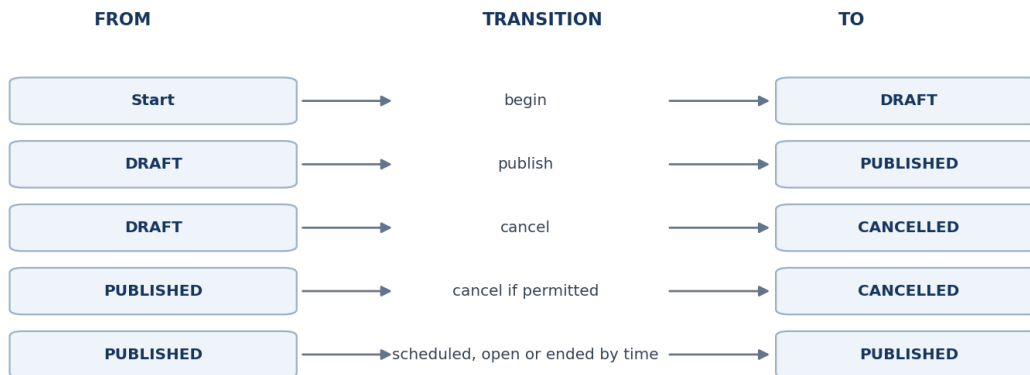


Objection activity diagram



State diagrams

The exam's open and closed periods are derived from time; the stored design status is draft, published or cancelled.





13 Testing strategy

Unit tests cover deadline calculation, score bounds, MCQ marking, objection eligibility and transition rules. Integration tests run services against an isolated PostgreSQL database and verify unique constraints, transactions and file metadata. API tests validate success, invalid input and error contracts. Frontend component and form tests exercise Persian and RTL display, authentication states, question navigation, upload feedback and guarded actions. End to end tests cover login through MCQ submission, automatic score, publication, objection and professor decision; a second path covers question download, answer upload, descriptive review, grading and publication.

Risk	Essential negative test
Broken access control	Student requests another student's attempt; professor requests another professor's exam, file or grade.
Time manipulation	Submit or upload at and after server cutoff; alter client clock or duration display.
Answer and grade exposure	Inspect pre submission question payload for correct flags; fetch unpublished grade.
Tampering and races	Send score in client request; replay final submit; issue concurrent saves and finalization.
Upload safety	Wrong extension or MIME, oversized payload, traversal filename and malformed file.
Objections	Wrong owner, unpublished grade, expired window, repeated or unauthorized resolution.

A passing test plan is not evidence that these tests were executed. Retain test results, environment versions and known failures as separate project evidence.

14 Deployment requirements

Provision a supported Node.js runtime, PostgreSQL, application secret values, secure private file storage and an HTTPS endpoint for frontend and API. Apply reviewed Prisma migrations, seed university accounts and course assignments through controlled administration, and configure allowed browser origins. Set the application time zone handling explicitly for Asia/Tehran conversion while storing absolute timestamps. Restrict storage and database access, configure backups and restore drills, collect operational logs without credentials, and establish a deadline processing mechanism that finalizes overdue attempts. Document the actual job runner and storage provider only after deployment configuration is inspected.

15 Future improvements

Potential later work includes university single sign on, explicit accommodation policies, notification delivery, stronger file scanning, automated deployment and backup verification, and broader accessibility review. These are proposals, not implemented features.

16 Conclusion

The system design provides a coherent route from university provisioned identity and course eligibility to exam participation, secure submissions, grade publication and accountable objection review. Its central controls are server enforced ownership, trusted deadlines, private files and preserved grade revisions. Before submitting this as a verified implementation report, reconcile the six month objection rule with code and migrations and attach repository based API and test evidence.